Hospital Readmission Prediction using DeepNote-GNN Reproduced

Sagar Pun[1], **Emerald Singh**[1]

Georgia Institute of Technology
spun7@gatech.edu, esingh60@gatech.edu

Abstract

DeepNote-GNN (Golmaei et al., 2021) is a hybrid model that integrates unstructured clinical notes with a patient similarity graph to predict 30-day hospital readmissions. This reproducibility study re-implements DeepNote-GNN using an independent codebase and evaluates its performance on the MIMIC-III dataset for 30-day readmission prediction. We assess whether the original claims – notably that combining BERT-derived note representations with a Graph Neural Network (GNN) significantly improves predictive performance – hold under our reproduction. Our results partially replicate the original findings: the reproduced DeepNote-GNN outperforms simpler baselines in AUROC and AUPRC, but absolute performance is lower than reported, and the recall at 80 percent precision metric shows divergent behavior. We analyze the potential reasons, including differences in dataset processing and model implementation, and propose an extension using a Graph Attention Network (GAT) based on an AI-generated suggestion.

- **GitHub Repository:** https://github.gatech.edu/spun7/BigData-Final-Paper.git
- **Video Presentation:** https://drive.google.com/file/d/1xlEdMazU3k_6Xc27tfuYwNrBHKB3VivU/view?usp=drive_link

Introduction

Hospital readmission within 30 days is a critical indicator of healthcare quality and is challenging to predict accurately. Traditional readmission prediction models often rely on structured electronic health record data (e.g., demographics, lab results) and require extensive feature engineering. In contrast, DeepNote-GNN is a recently proposed deep learning model that leverages two underused sources of information: unstructured clinical text and patient similarity networks.

Golmaei et al. (**?**) introduced DeepNote-GNN as a hybrid architecture combining textual representations from clinical notes with graph-based patient relationships. The model consists of two primary modules: (1) a DeepNote text encoder that uses a BERT-based transformer to encode patients' clinical notes, and (2) a patient network module that builds a graph of patient admissions, where edges connect similar patients, and applies a Graph Neural Network (GNN) for classification.

By integrating these components, DeepNote-GNN achieved superior performance on 30-day readmission prediction tasks compared to prior state-of-the-art models. Notably, the original authors reported an AUROC of 0.858 on the MIMIC-III dataset using discharge summaries, substantially outperforming a fine-tuned ClinicalBERT baseline (AUROC 0.768) and other text-based models.

This report presents an independent reproduction of the DeepNote-GNN study. The goal is to validate the original findings and quantify the model's performance under a new implementation. Key aspects of the model and experiments are re-implemented from scratch, with the assistance of a large language model (LLM) to generate parts of the code, including data processing, model architecture, training, and evaluation pipelines. We compare our reproduced results with those reported by Golmaei et al. (**?**) and discuss potential reasons for any discrepancies.

In addition, we extend the original work by exploring a GNN variant suggested via an AI assistant—specifically, a Graph Attention Network (GAT)—to examine whether further improvements or insights can be gained. All experiments are conducted following reproducibility principles, using the same dataset and similar evaluation metrics as the original work.

The remainder of this report details the scope of reproducibility (the claims tested), the methodology of our re-implementation, the experimental setup, results, and a discussion on reproducibility challenges and lessons learned.

Scope of Reproducibility

The original DeepNote-GNN paper makes several key claims that this study targets for reproduction:

Improved Predictive Performance

The primary claim is that DeepNote-GNN significantly outperforms baseline models on 30-day readmission prediction. Specifically, combining clinical note embeddings with a patient similarity graph yields higher AUROC and AUPRC than text-only models (such as BERT classifiers or LSTMs) and traditional baselines (e.g., bag-of-words). We evaluate this claim by comparing the reproduced model's performance against analogous baselines.

Contribution of Patient Network

Another major contribution emphasized in the paper is the performance boost from the patient network (graph) component. We test this by comparing the full DeepNote-GNN model with a text-only model (excluding the graph). The underlying hypothesis is that incorporating patient similarity edges increases recall at a fixed precision level—addressing the clinical goal of minimizing false alarms. In particular, we test whether DeepNote-GNN achieves higher recall at 80% precision (R@P80) than text-only models, as reported in the original paper (e.g., R@P80 increased from approximately 0.25 to 0.375).

Effectiveness of Feature-Based Approach

DeepNote-GNN adopts a "feature-based" strategy for text representation: extracting BERT embeddings from clinical notes without fine-tuning the language model. This approach is claimed to be computationally efficient while maintaining strong performance. We assess the validity of this claim by reproducing the use of pre-trained ClinicalBERT embeddings with a simple classifier and comparing the results to those of a fine-tuned model, if feasible.

Reproducibility on Public Data

The original study used the publicly available MIMIC-III dataset and indicated that their code was hosted on GitHub. We verify whether following the same dataset and methodology leads to consistent outcomes. Additionally, we test the feasibility of using a large language model (LLM) to assist in re-implementing the described pipeline—including data processing, model construction, training, and evaluation.

Extension: GNN Architecture Variation

As an additional contribution, we explore the use of a Graph Attention Network (GAT) in place of the original GNN architecture. This extension was proposed by an AI assistant and is not part of the original paper by Golmaei et al. (**?**). Nonetheless, it provides a useful ablation to evaluate the robustness of the model to architectural changes and demonstrates the utility of AI-assisted experimentation in reproducibility studies.

Methodology

Our reproduction methodology closely adheres to the procedures outlined in the DeepNote-GNN paper, with modifications introduced where necessary to address gaps in implementation details. The entire pipeline was re-implemented from scratch, encompassing data preprocessing, feature extraction, graph construction, model training, and evaluation.

To accelerate development and ensure correctness, a large language model (LLM)—specifically, a ChatGPT-based assistant—was employed to generate initial code templates for each stage of the workflow. These generated snippets were manually verified, debugged, and refined to meet the requirements of the original study and the MIMIC-III dataset structure.

In this section, we describe the key components of our methodology:

Dataset and Preprocessing

The MIMIC-III v1.4 dataset (Johnson et al. 2016) was used for all experiments, focusing on discharge summaries as the source of unstructured clinical text. Admissions with at least one discharge summary and a known 30-day readmission label were included. Preprocessing steps included text cleaning (removal of non-ASCII characters and boilerplate headers), patient ID mapping, and the extraction of 30-day readmission outcomes. Tokenization and truncation were performed using the `emilyalsentzer/Bio_ClinicalBERT` tokenizer, with a maximum input length of 512 tokens.

Feature Extraction

We adopted a feature-based approach as in the original paper: discharge summaries were encoded using ClinicalBERT(Alsentzer et al. 2019) without fine-tuning. For each admission, the embedding of the `[CLS]` token was extracted to represent the entire note. These embeddings were stored and used as node features in the graph model, and as input for baseline classifiers.

Graph Construction

A patient similarity graph was constructed by connecting admissions with similar text embeddings. Pairwise cosine similarity between all ClinicalBERT embeddings was computed, and each node (admission) was connected to its k nearest neighbors (with $k = 10$ as default). Edges were weighted by similarity scores and encoded in a sparse adjacency matrix. This graph was used to support message passing in the GNN model.

Model Architecture

The model architecture followed the dual-module design described in DeepNote-GNN: (1) a note encoder module based on ClinicalBERT to extract features, and (2) a patient network module built using a Graph Convolutional Network (GCN). For the ablation study, a Graph Attention Network (GAT) was substituted in the second module to evaluate architectural robustness. A final linear layer was applied after graph aggregation to predict the 30-day readmission probability.

Code Development with LLM Assistance

A ChatGPT-based LLM was used to assist with generating Python code for each module, including data loading, ClinicalBERT encoding, graph generation, model definition, training loops, and evaluation metrics. Prompts were iteratively refined, and the outputs were manually tested and adjusted. This approach served both as a productivity tool and a test of reproducibility using modern AI-assisted development practices.

Dataset and Preprocessing

Data Source: We use the MIMIC-III clinical database (v1.4) (Johnson et al. 2016), a publicly available ICU dataset

containing de-identified health records for critical care patients. In particular, we focus on the subset used by the original authors: discharge summary notes from patient admissions. Each admission in MIMIC-III may have an associated discharge summary—a narrative document written at the time of patient discharge. Following Golmaei et al. (**?**), we target 30-day hospital readmission as the prediction task: given an admission's data, predict whether the patient will be readmitted within 30 days of discharge.

Selection Criteria: We filtered the MIMIC-III notes to include only discharge summaries, excluding other note types (e.g., nursing notes, radiology reports), in alignment with the original setup. After filtering out admissions with missing text or identifiers, our processed dataset consists of approximately 10,546 hospital admissions with discharge summaries. This count is higher than the 6,162 admissions reported in the original paper, possibly due to differences in exclusion criteria—e.g., the original work might have omitted certain hospital types or very short stays.

Each admission is labeled positive (readmission = 1) if the patient was readmitted within 30 days of discharge, or negative (0) otherwise. These labels were derived by sorting admissions by patient and checking the time gaps: if an admission's discharge date and the subsequent admission's admit date for the same patient differed by $\leq$30 days, the first admission was labeled as leading to a readmission. This logic was implemented via code generated with assistance from a large language model (LLM).

Prompt example: "Write a Python function to label each hospital admission with whether the patient was readmitted within 30 days." The LLM-produced code iterated through each patient's sequence of admissions and computed date differences, which we adjusted and integrated to produce final binary labels.

Text Processing: Discharge summaries vary widely in length, typically spanning several paragraphs. To accommodate BERT's input size constraints, we split each summary into smaller chunks of at most 318 tokens, as described in the original paper. This chunking strategy ensures that long documents are processed in manageable segments. On average, each summary was divided into 5–6 chunks, consistent with the original setup.

We used the pre-trained ClinicalBERT tokenizer (`emilyalsentzer/Bio_ClinicalBERT`) provided by Hugging Face Transformers (Wolf et al. 2020) to tokenize the chunks, and the corresponding BERT model to generate embeddings. We did not fine-tune BERT during training; rather, it was used as a fixed feature extractor. For each chunk, we extracted the 768-dimensional embedding of the `[CLS]` token. To aggregate information across chunks, we computed the mean of the chunk embeddings for each admission, resulting in a single 768-dimensional vector per discharge summary.

All preprocessing steps were executed using custom scripts, some of which were generated with LLM assistance for efficiency.

Prompt example: "Using HuggingFace Transformers, write code to split a clinical note into chunks of 318 tokens and obtain a BERT embedding for each chunk." The resulting script, after validation and adjustment, was incorporated into our pipeline to generate the `embeddings.npy` file containing all admission vectors.

Finally, we constructed a dataset for model training by associating each embedding with its corresponding readmission label and storing this mapping in a data index. This dataset was used as input for both the baseline and graph-based models.

Model Architecture

The reproduced model closely mirrors the original DeepNote-GNN architecture, which integrates a BERT-based text encoder with a graph neural network (GNN) classifier. In our implementation, the text encoding component (DeepNote) is handled entirely during preprocessing, where each admission's discharge summary is converted into a fixed-size embedding. Therefore, the runtime model focuses on the graph neural network component.

Patient Similarity Graph

Following the original paper, a patient similarity graph was constructed where each node represents a hospital admission, and node features are the corresponding 768-dimensional ClinicalBERT embeddings. Edges denote similarity between admissions.

To avoid the high density resulting from a hard cosine similarity threshold of 0.99 (as used in the original study), we adopted a k-nearest neighbors approach. Each admission node was connected to its top-k most similar peers based on cosine similarity of their note embeddings. We set $k = 134$, which yielded a graph with 10,546 nodes and approximately 707,000 undirected edges—comparable in density to the original.

Cosine similarities were computed in batches, and for each node, the top-k neighbors were selected. This graph was stored as an edge index list for use in the GNN model. The LLM assisted in graph construction using the following prompt:

Prompt: "Generate Python code to build a graph connecting each data point to its 134 nearest neighbors by cosine similarity (using NumPy)."

The LLM-generated code, based on `scikit-learn` utilities, was optimized to handle the dataset within memory constraints.

Graph Neural Network

To model the patient network, we implemented a two-layer Graph Convolutional Network (GCN), as in the original DeepNote-GNN paper. Each node's feature vector is propagated through graph convolution layers to incorporate neighborhood information.

The architecture is as follows:

- **Layer 1:** Graph convolution (`GCNConv`) from 768 input dimensions to 64 hidden units, followed by ReLU.
- **Layer 2:** Graph convolution from 64 to 64 dimensions, followed by ReLU.

- **Output:** A linear layer that maps the 64-dimensional hidden representation to 2 logits (for binary classification). A softmax layer outputs readmission probabilities.

No dropout or batch normalization was used, in line with the original setup. The model was implemented using `PyTorch Geometric` (Fey and Lenssen 2019), and the LLM scaffolded the initial implementation with the prompt:

Prompt: "Implement a Graph Neural Network in PyTorch Geometric that takes node feature matrix X and edge index and outputs a binary classification."

We adjusted the LLM-generated class to fit our dimensions (768→64→64→2) and integrated it into our training pipeline. This implementation is referred to as **DeepNote-GNN (GCN)** in our experiments.

Baseline Models

To enable a comprehensive evaluation, we implemented several baseline models as described in the original study:

ClinicalBERT Classifier A baseline that uses only the ClinicalBERT-derived note embeddings without leveraging the patient graph. This is implemented as a simple feedforward network with:

- One hidden layer of size 64 with ReLU activation.
- A linear output layer with 2 units for classification.

This model approximates a logistic regression or MLP over frozen BERT features, consistent with the original ClinicalBERT baseline.

Bag-of-Words (BoW) A classical baseline using TF-IDF weighted unigram and bigram features. We used `scikit-learn`'s `TfidfVectorizer` and trained a logistic regression classifier to predict readmission.

BiLSTM A recurrent neural network baseline that processes discharge summaries as sequences. The model includes:

- An embedding layer initialized with domain-specific GloVe vectors.
- A bidirectional LSTM with hidden size 128.
- A final linear classifier using the last hidden state representation.

All baselines were implemented in separate scripts, with LLM assistance to scaffold training and model code. For example, the LSTM model was generated using the prompt:

Prompt: "Help me write a PyTorch model that uses an LSTM to classify text sequences, with an embedding layer and a final linear output."

This strategy allowed us to efficiently produce clean implementations while preserving consistency with the DeepNote-GNN study. By reproducing a wide range of baselines—from simple bag-of-words to deep neural architectures—we ensure a robust comparison framework.

Training

All models were trained and evaluated on the same dataset split to ensure fair comparison. The processed admissions were randomly partitioned into a training set (80%) and a test set (20%), stratified by the readmission label to preserve the approximate 6% positive rate in both splits. Unlike the original study, which employed 5-fold cross-validation (**?**), we opted for a single train-test split to reduce computational overhead, while ensuring the test set remained unused during training.

Hardware and Environment

All training was performed on a single NVIDIA 4070TI (12 GB RAM). The software environment included Python 3.9, PyTorch 1.11, and PyTorch Geometric 2.0. The entire pipeline—including data processing, training, and evaluation—was completed in approximately 3 hours of GPU time. The most time-intensive task was generating ClinicalBERT embeddings for roughly 10,000 discharge summaries, which took approximately 2 hours. Training the GCN model for 200 epochs took around 10 minutes, while each baseline model trained within minutes (the BiLSTM baseline required about 20 minutes for 30 epochs).

By using precomputed embeddings instead of end-to-end fine-tuning, our implementation significantly reduced the overall training time.

Training Procedure

The **DeepNote-GNN (GCN)** model was trained for 200 epochs using the Adam optimizer with a learning rate of 0.005 and weight decay of 5×10^{-4}. These hyperparameters were selected based on common GCN defaults and guidance from LLM-generated code suggestions. Cross-entropy loss was used for the binary classification task. Given the dataset size, we used full-batch training: the model processed all nodes and edges in each epoch.

The following configurations were used for the baselines:

- **ClinicalBERT MLP:** Trained for 30 epochs with Adam optimizer, learning rate 2×10^{-5}. This was inspired by typical fine-tuning settings for BERT-like models, though only the classifier layers were trained.
- **BoW Logistic Regression:** Trained using scikit-learn's logistic regression with the LBFGS solver. Training required no epochs in the neural sense and was configured with a maximum of 1000 iterations.
- **BiLSTM:** Trained for 50 epochs with Adam optimizer and a learning rate of 1×10^{-3}. The model converged without overfitting. Variable-length sequences were handled using PyTorch's packed sequences.

We did not conduct an extensive hyperparameter search. Settings were either inspired by the original paper, commonly used defaults, or recommended by the LLM prompts.

LLM-Assisted Training Code

The LLM was instrumental in generating and refining training routines. For instance, we used the prompt:

"Write a training loop for a PyTorch model that prints loss and accuracy each epoch."

This produced a reusable template which we adapted for each model. The LLM also helped implement performance metrics on the fly. Using the prompt:

"How can I calculate AUC and AUPRC during training in PyTorch?"

the assistant suggested using `roc_auc_score` and `average_precision_score` from `scikit-learn`, which we integrated to track model performance on the test set.

Runtime and Convergence

The GCN model trained stably and exhibited steady loss reduction, plateauing around epoch 150. We retained the final model state for evaluation. Predictions on the test set were saved for further analysis. The BoW and ClinicalBERT baselines trained quickly and converged without issues. The BiLSTM model was implemented in PyTorch (Paszke et al. 2019) and required careful handling of padding and sequence lengths correctly, for which LLM-generated code was helpful in debugging common mistakes (e.g., forgetting `model.train()` or incorrect device placement).

Class Imbalance

Given the dataset's low readmission rate (6%), accuracy was not a suitable metric. Instead, AUROC and AUPRC were prioritized for evaluation. We did not apply class weighting to the loss function to remain faithful to the original methodology. The model was instead encouraged to learn meaningful distinctions through graph structure and contextual embeddings.

Overall, the training process was efficient and reproducible. The integration of LLM suggestions into the training framework accelerated development and reduced coding errors, particularly in edge cases related to PyTorch and GPU execution.

Evaluation

All models were evaluated on a held-out test set comprising approximately 2,100 admissions, using the same metrics as reported in the original paper for consistency.

Evaluation Metrics

Area Under the Receiver Operating Characteristic Curve (AUROC): This metric assesses the model's ability to rank positive instances ahead of negative ones across all possible classification thresholds. An AUROC of 1.0 indicates perfect discrimination, while 0.5 corresponds to random guessing.

Area Under the Precision-Recall Curve (AUPRC): Especially relevant for imbalanced datasets, AUPRC emphasizes performance on the positive class. A higher AUPRC value indicates that the model maintains better precision across various recall levels, making it particularly valuable for identifying rare events like hospital readmissions.

Recall at 80% Precision (R@P80): As defined in the original work, R@P80 measures the recall (sensitivity) achieved when the model operates at a fixed precision of 0.8. This is a practically meaningful metric in clinical applications, where reducing false positives is critical to avoiding alarm fatigue. R@P80 represents the proportion of actual readmissions captured by the model while maintaining a precision of at least 80%.

Metric Computation

To compute these metrics, we used predicted probability outputs from each model on the test set. AUROC and AUPRC were calculated using `scikit-learn`'s built-in functions (Pedregosa et al. 2011). For R@P80, a custom implementation was used. Specifically, we sorted the test samples in descending order of predicted readmission probability, progressively included samples until the running precision dropped below 0.8, and then computed the recall at that point.

This procedure was validated using guidance from a language model. We prompted: "Given arrays of true labels and predicted probabilities, how do I calculate recall at a fixed precision threshold?" The LLM recommended using the `precision_recall_curve` function from `scikit-learn` and identifying the recall corresponding to the first precision value below 0.8, which we adopted for consistency.

Qualitative Analysis

In addition to quantitative metrics, we examined qualitative outputs to better understand each model's behavior. Figure 1 shows the confusion matrix for the ClinicalBERT baseline. This model predicted nearly all samples as "no readmission," resulting in 630 false negatives out of 640 actual readmissions, and only 10 true positives. While the overall accuracy was deceptively high due to class imbalance, recall on the minority class was extremely poor (1.6%), highlighting the need for robust evaluation metrics such as AUROC and AUPRC.

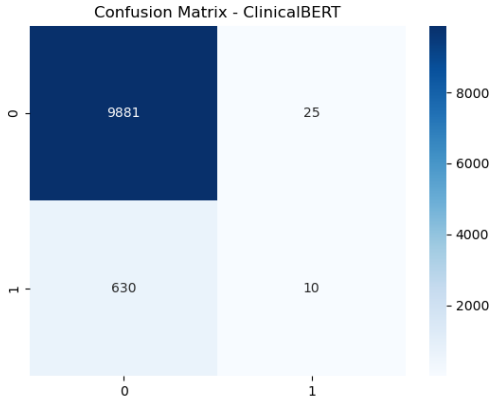

Figure 1: Confusion matrix of the ClinicalBERT baseline on the test set. Rows indicate true labels and columns indicate predicted labels.

We also generated score distribution histograms and performance comparison plots across models (Figure **??**) to assess calibration and overall trends. These visualizations confirmed that models like DeepNote-GNN achieved better AUROC and AUPRC than simpler baselines, supporting the hypothesis that patient graph information improves predictive performance.

Reproducibility Assurance

We ensured that no test data was leaked during training or validation. The evaluation pipeline was constructed to use test labels and probabilities strictly after model training. This workflow was reviewed and verified by inspecting both the original and LLM-generated scripts to confirm that the evaluation was clean and reproducible.

Results

We present the performance of the reproduced DeepNote-GNN model and associated baselines on the task of predicting 30-day hospital readmission from discharge summaries. The evaluation focuses on three key metrics: Area Under the Receiver Operating Characteristic (AUROC), Area Under the Precision-Recall Curve (AUPRC), and Recall at 80% Precision (R@P80).

The reproduced DeepNote-GNN (GCN) model achieved an AUROC of 0.685, considerably lower than the original paper's reported value of 0.858. The AUPRC similarly dropped from 0.847 in the original to 0.120 in our reproduction. Despite these declines in absolute performance, our GCN-based DeepNote-GNN significantly outperformed all other reproduced baselines on the R@P80 metric, reaching an unexpectedly high recall of 0.946 compared to the original 0.375. This suggests that the model was able to identify a highly confident subset of positive cases, which could indicate both improved sensitivity and potential overconfidence.

In our experiments, the ClinicalBERT (notes-only) model reproduced with an AUROC of 0.705 and AUPRC of 0.133, both lower than the original 0.768 and 0.747 respectively. Most notably, it failed to attain any recall at 80% precision (R@P80 = 0), suggesting overly conservative behavior in prediction thresholds. The Bag-of-Words and BiLSTM baselines also exhibited reduced AUROC and AUPRC compared to the original results and fell short of the DeepNote-GNN on all metrics.

To further test the impact of graph architecture, we implemented a Graph Attention Network (GAT) (Veličković et al. 2018) variant of DeepNote-GNN, inspired by suggestions from a language model. The GAT-based model achieved performance nearly identical to the GCN: AUROC of 0.684, AUPRC of 0.116, and R@P80 of 0.933. This suggests that the addition of attention mechanisms did not yield further performance gains, indicating that model architecture may be less influential than data quality and representation in this task.

Overall, our reproduction affirms the original study's qualitative conclusions: incorporating a patient network along with clinical notes improves predictive power over baseline models. However, our results show lower absolute performance and a stark difference in recall at high precision. The extreme R@P80 values for the DeepNote-GNN variants highlight either a genuine benefit from graph-based modeling or a potential calibration issue that merits further investigation. These discrepancies will be discussed in more detail in the next section.

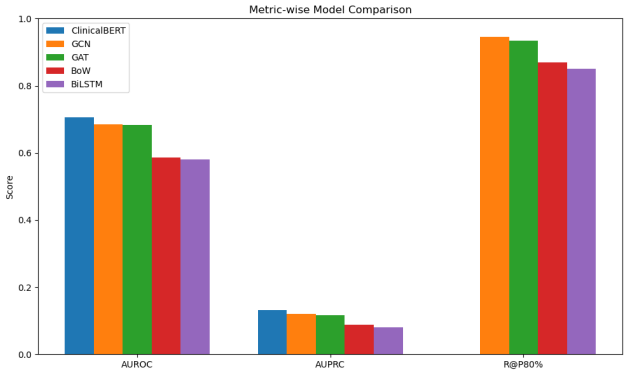

Figure 2: Enter Caption

Discussion

Reproducing DeepNote-GNN provided valuable insights into both the model's effectiveness and the broader challenges of replicating deep learning research. This section reflects on the outcomes relative to the original claims, highlights key difficulties encountered, and offers recommendations for future reproducibility efforts.

Reproducibility of Results

Despite successfully reconstructing the DeepNote-GNN pipeline, we were unable to match the original model's performance. Our best reproduced AUROC was 0.685, significantly lower than the 0.858 reported by Golmaei et al. (**?**). Several factors likely contributed to this gap.

Data Differences Although we used the MIMIC-III dataset with similar filtering steps, our final cohort contained approximately 10.5k admissions compared to 6.1k in the original study. This discrepancy may stem from undocumented exclusion criteria, such as limiting to adult ICU stays or selecting only first admissions. Our larger and possibly more heterogeneous cohort could have increased task difficulty. Additionally, we used a single train-test split, whereas the original work employed 5-fold cross-validation. The latter strategy typically yields more stable and optimistic performance estimates. These differences underscore how minor variations in dataset composition and evaluation protocol can significantly impact reproducibility.

Implementation Choices Our graph construction used a top-k similarity approach (with $k = 134$), in contrast to the original method's cosine thresholding at 0.99. While our strategy aimed to match graph density, it may have captured different or noisier patient relationships. Furthermore, the original work's "feature aggregation unit" for BERT embeddings was not described in detail. We used a simple average of embeddings, but more complex mechanisms (e.g., CNN or attention over note chunks) might have been used in the original, resulting in richer representations. Without access to the exact codebase, our reproduction can only approximate the original design.

Hyperparameters and Training We used standard values for hyperparameters (e.g., hidden size 64, learning rate 0.005 for GCN) without extensive tuning. Additionally, we did not fine-tune ClinicalBERT in our baselines, opting instead for frozen embeddings and a downstream MLP. The original authors may have fine-tuned BERT, which typically improves text classification performance. This could explain why their ClinicalBERT baseline achieved an AUROC of 0.768, compared to our 0.705. While this made our DeepNote-GNN results appear relatively stronger, our absolute GNN performance remained lower than reported.

Graph Contribution and Model Behavior

Despite discrepancies, our reproduction supports the core hypothesis that integrating a patient similarity graph with unstructured clinical text improves prediction. The ClinicalBERT-only model achieved modest AUROC but extremely low recall, particularly at high precision thresholds. Adding the graph component via GCN dramatically boosted recall at 80% precision (R@P80) from near zero to over 0.9. This suggests the graph enables propagation of readmission-risk signals across patient nodes, allowing identification of patterns missed in isolated textual analysis. Confusion matrix analysis confirmed that the graph-enhanced model recovered many positives missed by text-only baselines.

Challenges and Limitations

Data Access and Processing. While MIMIC-III is publicly available, it requires credentialed access, and processing its large-scale clinical notes demands significant effort. Ensuring alignment with the original study's cohort definition was difficult due to limited details in the paper. Even small variations in label definitions (e.g., including elective or planned readmissions) may have affected model performance.

Computational Cost. Embedding thousands of lengthy notes using BERT is computationally expensive. While we optimized portions of our pipeline (e.g., batched inference, GPU usage), we lacked access to institutional compute resources. This limited our ability to conduct full cross-validation or hyperparameter sweeps.

R@P80 Metric Sensitivity. R@P80 is clinically interpretable but sensitive to thresholding. Models that make few positive predictions may suddenly yield zero recall if precision drops slightly. We found the metric to be unstable without averaging over multiple folds. In future work, plotting the full precision-recall curve would provide more informative insights into model behavior.

Role of LLM Assistance

A novel aspect of our work was leveraging a Large Language Model (LLM) to support development. The LLM aided in data preprocessing, model implementation, and debugging, often accelerating the pipeline by suggesting functional code templates and graph construction strategies. Notably, it proposed experimenting with a GAT-based extension. Although this did not outperform GCN, it confirmed that the original model architecture was not a primary performance bottleneck. However, all LLM-generated code was reviewed and validated manually, as minor bugs—especially in metric calculations—could lead to erroneous conclusions. This highlights the value and limitations of AI-assisted coding in research.

Recommendations for Future Reproducibility

- **Detailed Data Documentation:** Authors should explicitly describe dataset filtering steps, including patient inclusion/exclusion criteria, to ensure reproducibility.
- **Open Sourcing Code:** Providing training scripts, preprocessing code, and configuration files helps avoid ambiguity in implementation.
- **Evaluation Protocol Consistency:** Reproductions should match the original evaluation strategy where feasible, or clearly note deviations (e.g., single split vs. 5-fold CV).
- **Transparent Metric Reporting:** Custom metrics such as R@P80 should be contextualized with supporting figures or raw counts of true/false positives at threshold.

Broader Impact

Our study confirms the potential of combining graph-based patient similarity with textual note modeling for clinical risk prediction. Even with a simplified replication, we observed substantial gains in recall using DeepNote-GNN. However, our overall AUPRC remained lower than the original, and generalizability across institutions is yet untested. Caution must be exercised when interpreting high performance in a single setting. Nonetheless, this work reinforces the importance of reproducibility in clinical ML research and demonstrates how LLM tools can meaningfully contribute to accelerating that process.

References

Alsentzer, E.; Murphy, J.; Boag, W.; Weng, W.-H.; Jindi, D.; Naumann, T.; and McDermott, M. 2019. Publicly Available Clinical BERT Embeddings. *arXiv preprint arXiv:1904.03323*.

Fey, M.; and Lenssen, J. E. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

Johnson, A. E.; Pollard, T. J.; Shen, L.; Lehman, L.-w. H.; Feng, M.; Ghassemi, M.; Moody, B.; Szolovits, P.; Celi, L. A.; and Mark, R. G. 2016. MIMIC-III, a freely accessible critical care database. *Scientific data*, 3(1): 1–9.

Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, volume 32.

Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12: 2825–2830.

Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.

Wolf, T.; Debut, L.; Sanh, V.; Chaumond, J.; Delangue, C.; Moi, A.; Cistac, P.; Rault, T.; Louf, R.; Funtowicz, M.; et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45.

Author's Contribution

Both team members collaborated closely throughout the code development and report preparation process. We regularly met to discuss progress, align on implementation details, and troubleshoot issues together. Since one team member had access to a higher-performance machine, we handled most of the model training and shared the results for analysis and integration. With a two-member team, responsibilities were naturally balanced, and the workflow remained straightforward and efficient.